POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

IoT Challenge 3
Part 1: Node-Red

Tesi di Laurea Magistrale in
Telecommunication Engineering

Author: **Hong CHEN** [redacted]
Mengzhen LI [redacted]

Student ID: [redacted] & [redacted]
Advisor: Prof. Name Surname
Co-advisors:

Academic Year: 2025-04

Contents

1 | Part 1: Challenge Node-Red

PART 1 – Challenge: Overall Flowchart of Node-RED as shown in Figure 1.1.
Description of Nodes:

- **Time Inject Node:** Used to periodically trigger the entire system, set to trigger every 5 seconds.
- **read/write file:** Responsible for generating and reading/writing specific target files, such as `id_log.csv`.
- **MQTT out/in:** Used to publish or subscribe messages to/from specified topics.
- **http request:** Uploads specific data (e.g., the number of ACKs) to the Thingspeak platform.
- **function node:** Implements the detailed logic for each processing step, such as data extraction, filtering, and calculation.
- **delay:** Used to control the data transmission rate, set to 4 messages per minute in this system.
- **csv node:** Converts the contents of challenge3.csv into a standard object format for subsequent processing.
- **chart node:** Visualizes the output data in chart form, and users can view the real-time charts by accessing `http://localhost:1880/ui`.
- **debug:** Monitors and outputs intermediate data in real-time, making it easier to debug and verify the flow.

1.1. W1: Random ID Generation and Periodic MQTT Publishing

The goal of the first part is to implement a periodic ID generation system that performs the following operations:

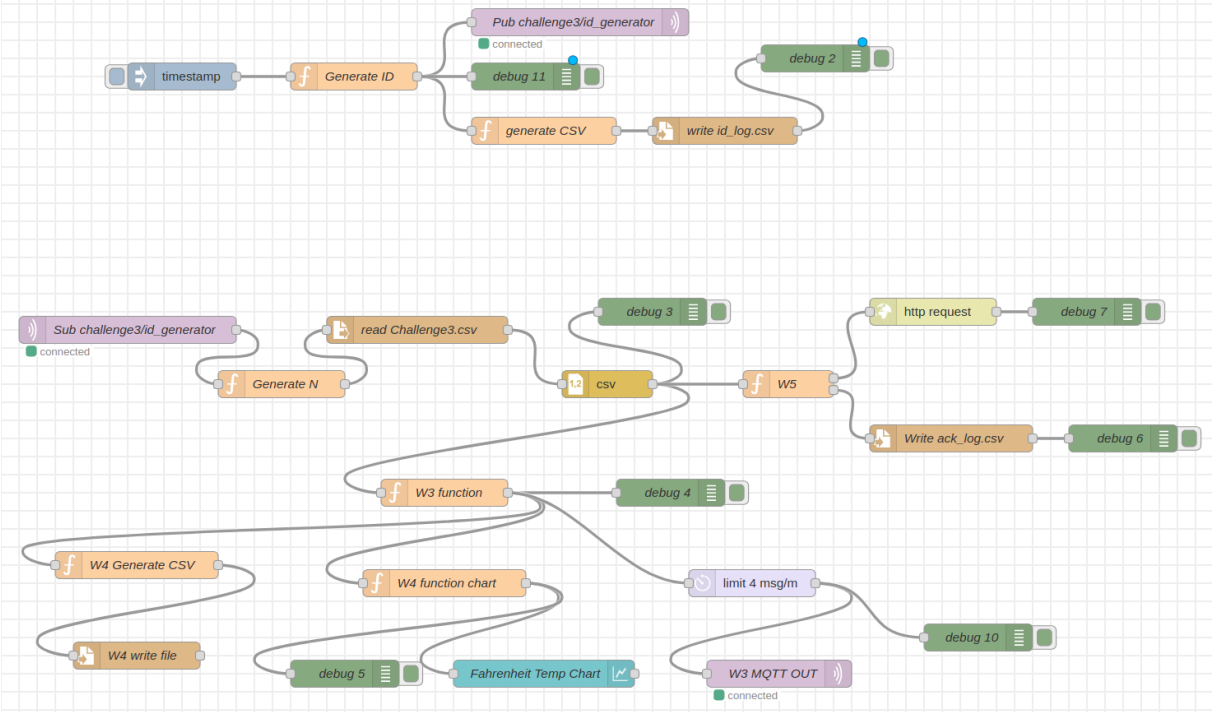

Figure 1.1: Part1: Node-Red Flow

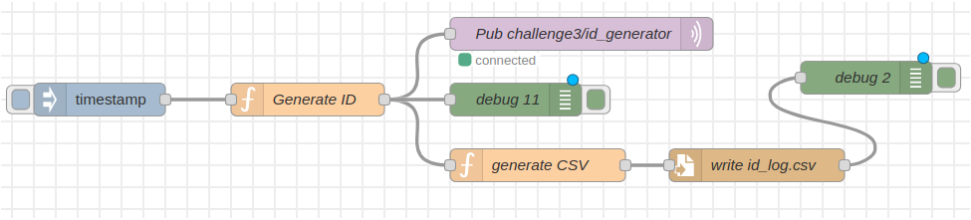

Figure 1.2: W1:Generate ID(Pub) and csv file

Every 5 seconds, generate a random ID in the range [0, 30000].
Retrieve the current UNIX timestamp.
Construct a JSON message containing the generated ID and timestamp.
The Node-RED flow for Step 1 is shown in Figure 1.2:
Publish the JSON message to the following MQTT broker and topic:

```
Broker: mqtt://localhost:1884
Topic: challenge3/id_generator
```

Append each message's ID and timestamp to a local CSV file named `id_log.csv` for record-keeping.
Example JSON Payload as shown in Figure 1.3,:

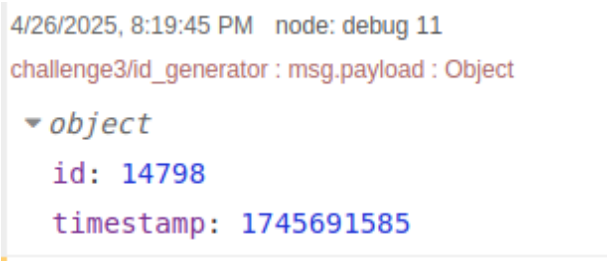

Figure 1.3: W1-Log:Pub ID to topic

1.2. W2: Subscription and Message Handling from Topic

This task involves subscribing to the MQTT topic used by the random ID generator, and performing the following operations upon receiving each message:

- **Receive JSON Message:** Subscribe to the topic `challenge3/id_generator` to receive messages in JSON format.
- **Extract and Process ID:** Parse the JSON message to extract the id field, then compute the value **N = id % 7711**.
- **Retrieve Corresponding Row from CSV:** Use the computed N value as a frame index to retrieve the corresponding row from the `challenge3.csv` file.

1.3. W3: Identifying and Re-Publishing MQTT Publish Messages

The objective of this part is to check whether the retrieved row from the CSV file corresponds to an MQTT Publish message. If so, the topic and payload fields are extracted and the message is re-published in the specified format.
The published message should be in JSON format, structured as follows:

```
"timestamp": CURRENT_UNIX_TIMESTAMP,
"id": SUB_ID,
"topic": MQTT_PUBLISH_TOPIC,
"payload": MQTT_PUBLISH_PAYLOAD
```

In cases where a packet contains multiple Publish messages, each Publish is extracted and sent as an individual MQTT publish message. Additionally, if any portion of the payload is missing or incomplete, it is treated as an empty payload during the processing stage.
The following snippet presents the key part of the implementation.

```
var topicIndex = 0;

messages.forEach(function(message) {
var topicMatch = message.match(/\[([^\]]+)\]/);  // extract [topic]
if (topicMatch) {
    var topic = topicMatch[1];
    var entry = {
        timestamp: new Date().toISOString(),  // Use current timestamp
        id: N,
        topic: topic,
        payload: payloads[topicIndex] || null
    };
    node.warn("Matched topic: " + topic + " with payload: " + entry.payload);
    results.push(entry);
    topicIndex++;
} else {
    node.warn("No topic match found in message: " + message);
}
```

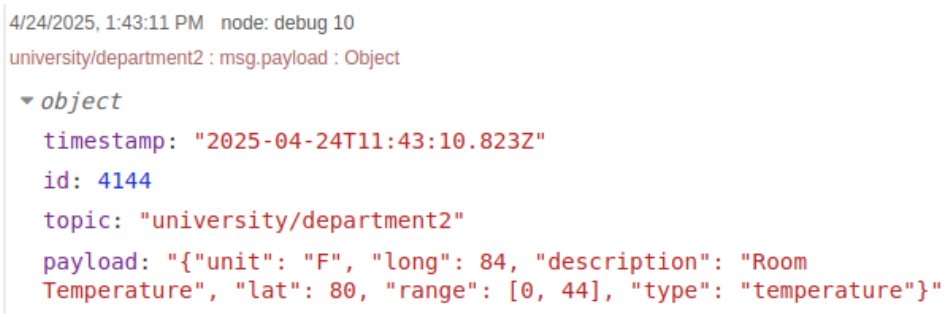

Figure 1.4: W3:Topic: "university/department2", Pub

To avoid message flooding, the system must enforce a maximum publishing rate of 4 messages per minute, implemented using a rate limiter.
An example of the message structure published in Step 3 is shown in Figure 1.4.

1.4. W4: Extracting Temperature Messages

This task focuses on filtering and processing valid temperature messages from MQTT Publish data for visualization and storage.
Parse the payload field of each message as JSON, and check whether it meets the following criteria:

- Must contain "type": "Temperature"
- Must contain "unit": "F"

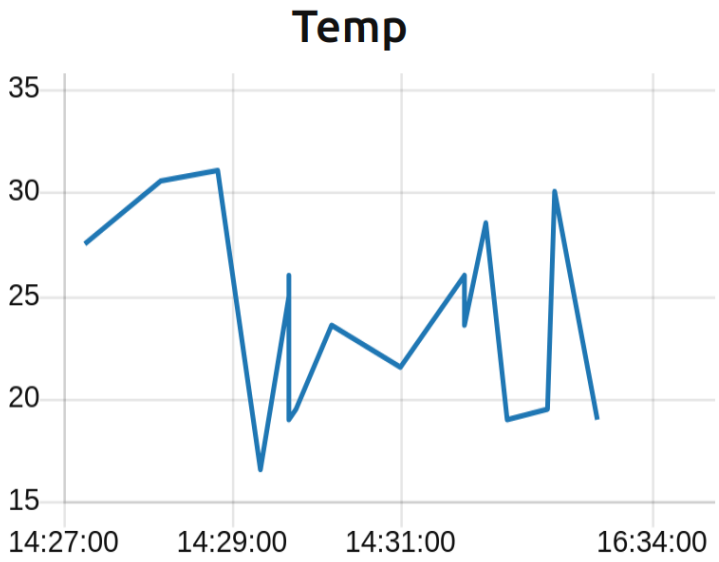

Figure 1.5: Temp Line Chart

Temperature Calculation:
If the message meets the criteria, extract "range": "min": X, "max": Y and compute the average temperature as:

```
MEAN_VALUE = (X + Y) / 2
```

The resulting temperature values are plotted in real-time using the Chart node in Node-RED, forming a temperature trend curve.
The generated temperature line chart is shown in Figure 1.2.
All valid temperature messages should also be written to a CSV file named `filtered_pubs.csv`, with the following format:

```
No., LONG, LAT, MEAN_VALUE, TYPE, UNIT, DESCRIPTION
```

As shown in file `filtered_pubs.csv` , when the global counter counter reaches 80, a total of 10 records are generated and stored in the file with this structure.

1.5. W5: Processing MQTT ACK Messages

This part involves processing MQTT ACK messages, where if the message has Frame No. = N and is of an MQTT ACK type (e.g., Publish Ack, Connect Ack, Sub/Unsub Ack), a global ACK counter is incremented. The relevant information of the message is then saved into a CSV file named `ack_log.csv`.
CSV File Format:

```
No.: Row number, incremental
TIMESTAMP: The current time
SUB_ID: The Message ID
MSG_TYPE: The message type (e.g., Connect Ack)
```

Once the ACK message is saved, the global ACK counter value is sent to the `ThingSpeak` channel via HTTP API, passing the ACK counter value into the channel field. The specific ThingSpeak visualization image link can be found in Section 1.6 of the report.

1.6. W6

In Step 6, for all cases where Frame No. = N does not contain an ACK or a Publish message, the program should ignore the message.
Additionally, the program should ensure that it stops processing after receiving exactly 80 ID messages from the subscription, and no more than 80 ID messages should be processed. Discarded messages (i.e., messages that are neither Publish nor ACK) should be counted towards the 80-message limit, ensuring the total does not exceed 80 valid messages. Once 80 valid ID messages have been received, the program should stop further message processing and should not process any subsequent ID messages.
Among the 80 messages within the limit, 10 messages met the ACK filtering criteria from Step 5. The ThingSpeak visualization is shown in Figure 1.3, and the specific link can be accessed here!

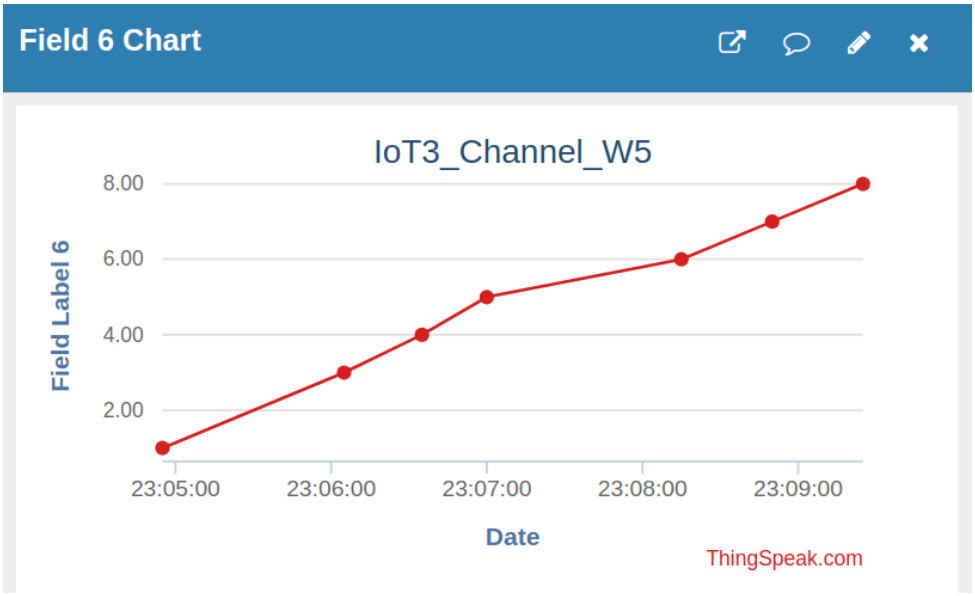

Figure 1.6: W6: Ack number of 80 messages limitation

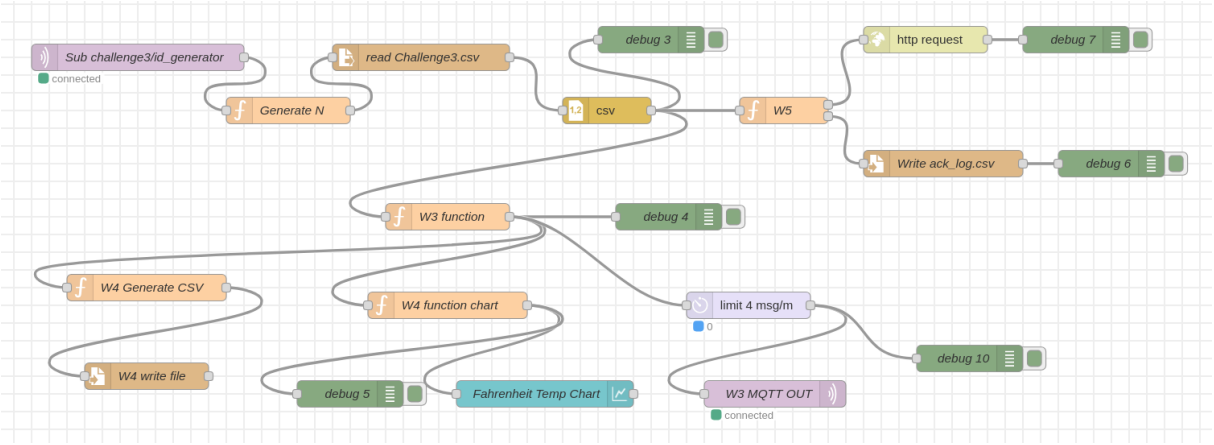

Figure 1.7: Node-red Flow